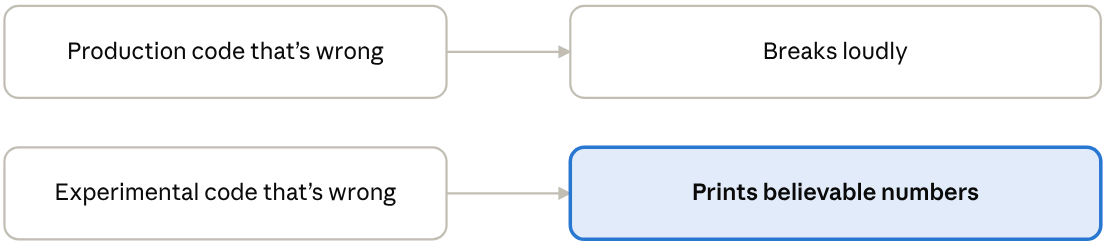
In the Age of Vibe Coding, Test Your AI Agent with MAT

Multi-Arm Testing: how six versions of one request showed us that our obvious suspects were innocent

In the age of vibe coding, writing code is the easy part. You describe what you want, an AI assistant writes it, it runs, and the numbers look plausible. That last part is the trap. Production code that's wrong usually breaks loudly. Experimental code that's wrong just prints believable numbers.

Wrong production code is loud; wrong experimental code is quiet



loud vs quiet failure · 2 lanes

Our AI agent was built that way. We described what we wanted, an AI assistant wrote most of it, and within weeks we had an agent running on Amazon Bedrock. Vibe coding got us there fast. Finding what was slow needed something slower and more careful.

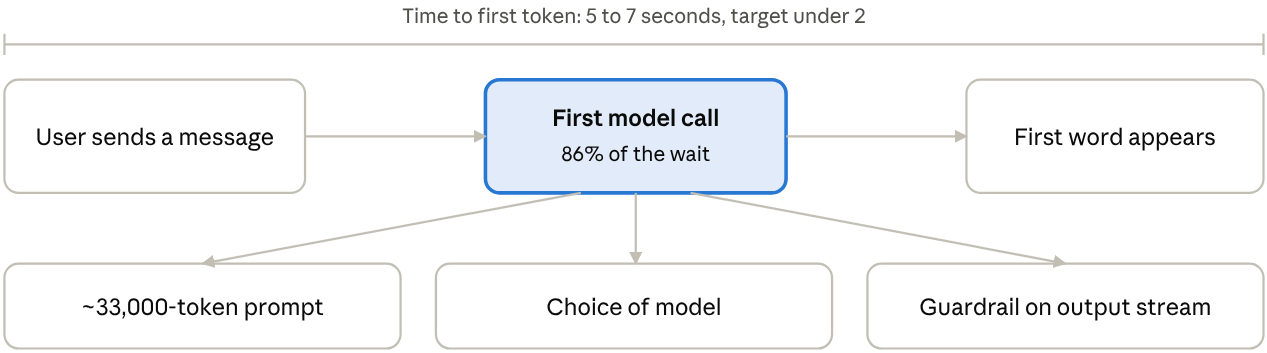
This is the story of one experiment: six versions of the same request, and what they showed about where our agent's first words were going.

The story

Users started saying the same thing: "It just sits there." The agent runs on Amazon Bedrock. Claude Sonnet does the thinking, it can call tools such as a content search, and Bedrock Guardrails check every reply.

The logs agreed with the users. Time to first token (TTFT), the time from sending a message to seeing the first word, was 5 to 7 seconds. Our target was under 2. The first model call took 86% of that wait.

The first model call is 86% of the wait, and it holds three suspects



Any of the three could be the culprit.

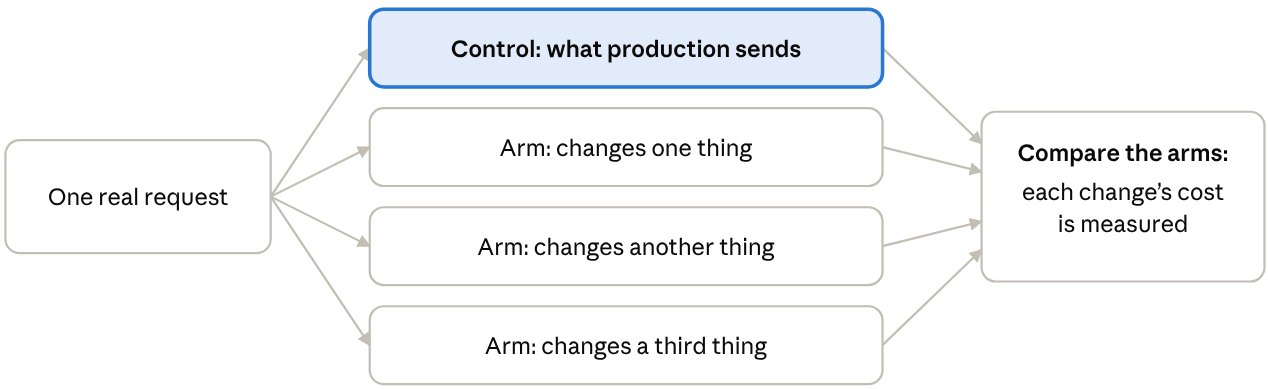
where the wait sits · 1 model call, 3 suspects

But “the model call is slow” isn’t a diagnosis. A model call is a bundle of decisions, and any of the three above could be the culprit. To find the problem, we needed Multi-Arm Testing (MAT).

What MAT is

MAT is controlled experiments for an agent.

- Take one real request your agent makes and replay it many times, in several versions called arms.
- One arm, the control, is exactly what production sends.
- Every other arm changes one thing.
- Compare the arms, and each one-thing change has its cost measured.

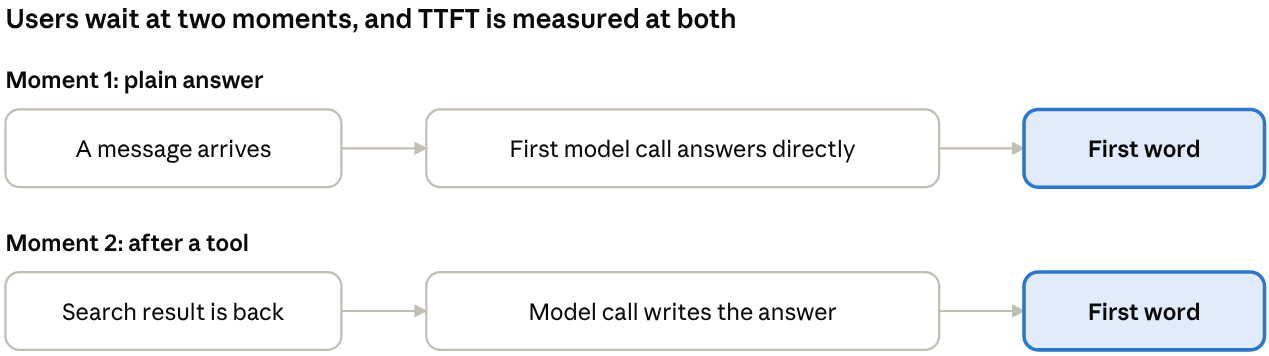


MAT layout · 1 request, control + arms

Step 1: Pick the target and the moments to measure

Our target was TTFT, measured at the two moments users wait for most:

- Plain answer: the agent's first model call, when it answers directly.
- After a tool: the model call that writes the answer once a search result is back.



the two measured moments · 2 lanes

Step 2: Build the control from the agent itself

The most common MAT mistake is hand-writing the "production" request. Don't. Run your agent's own request builder, including any routing (ours picks one of two guardrail profiles, based on a small Haiku classifier's label), and capture the request just before it would be sent.



control construction · do vs don't

```
def build_request(ask, page, model=MAIN_MODEL):
    """Run the agent's real request builder for this ask and page.
    Capture the Bedrock request instead of sending it.
    Includes routing, guardrail profile, tools, cache points."""
    ...
```

Step 3: Define the arms

These are the six arms we used. Each one answers one question.

Highlighted cells are the only differences from the control

Arm	Prompt	Guardrail	Model
control	Full prompt	Routed profile	Sonnet
other_guardrail	Full prompt	**Other profile**	Sonnet
guardrail_off	Full prompt	**Off**	Sonnet
small_prompt	**One line, no tools**	Routed profile	Sonnet
small_prompt_no_gr	**One line, no tools**	**Off**	Sonnet
fast_model	Full prompt	Routed profile	**Haiku**

the six arms · what each changes from the control

Arm	Question it answers
control	What do users get?
other_guardrail	Does the profile matter?
guardrail_off	What does the guardrail cost?
small_prompt	What does the big prompt cost?
small_prompt_no_gr	The model's own floor
fast_model	Would a faster model fix it?

```
def arms_for(ask, page):
    control = build_request(ask, page)
    small = shrink_prompt(control)       # one-line system prompt, no tools,
same ask
    return {
        "control":            control,                                  # what
users get
        "other_guardrail":    swap_guardrail_profile(control),          # does
the profile matter?
        "guardrail_off":      without_guardrail(control),               # what
does the guardrail cost?
        "small_prompt":       small,                                    # what
does the big prompt cost?
        "small_prompt_no_gr": without_guardrail(small),                 # the
model's own floor
        "fast_model":         build_request(ask, page, model=HAIKU),    # would
a faster model fix it?
    }
```

The arm that pays off most is small_prompt_no_gr. It shows how fast the model can be at all. Every other arm is read against that floor.

Step 4: Time each run, and record what produced it

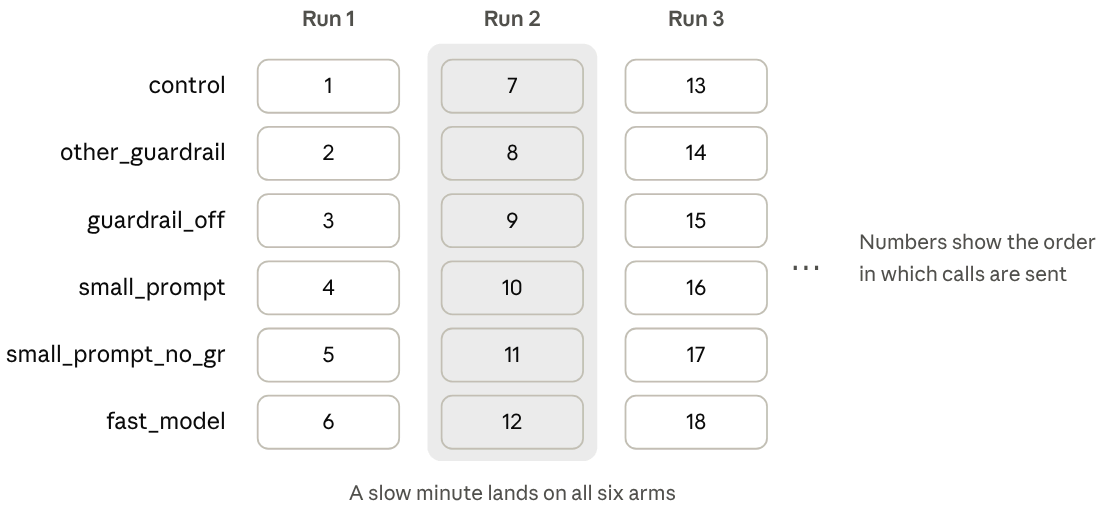
Each request is streamed once and timed. The client has retries off, so a throttled call is an error, not a slow run.

```
def time_call(client, request):
    """Stream one request. Return client marks and server timing.
    The client has retries off, so a throttled call is an error, not a slow
run."""
    row = {"first_text_ms": None, "total_ms": None, "stop": None,
           "server_latency_ms": None, "output_tokens": None, "error": None}
    ...  # mark first text delta, messageStop reason,
metadata.metrics.latencyMs
    return row

def run_session(asks, runs):
    rows = []
    for _ in range(runs):                       # round-robin: a slow minute
hits every arm
        for ask, page in asks:
            for arm, request in arms_for(ask, page).items():
                row = time_call(CLIENT, request)
                row.update(arm=arm, ask=ask, **provenance())  # commit, clean
tree, settings
                rows.append(row)
    return rows
```

The loop is round-robin: every run sends all six arms before the next run starts.

Round-robin order means a slow minute hits every arm



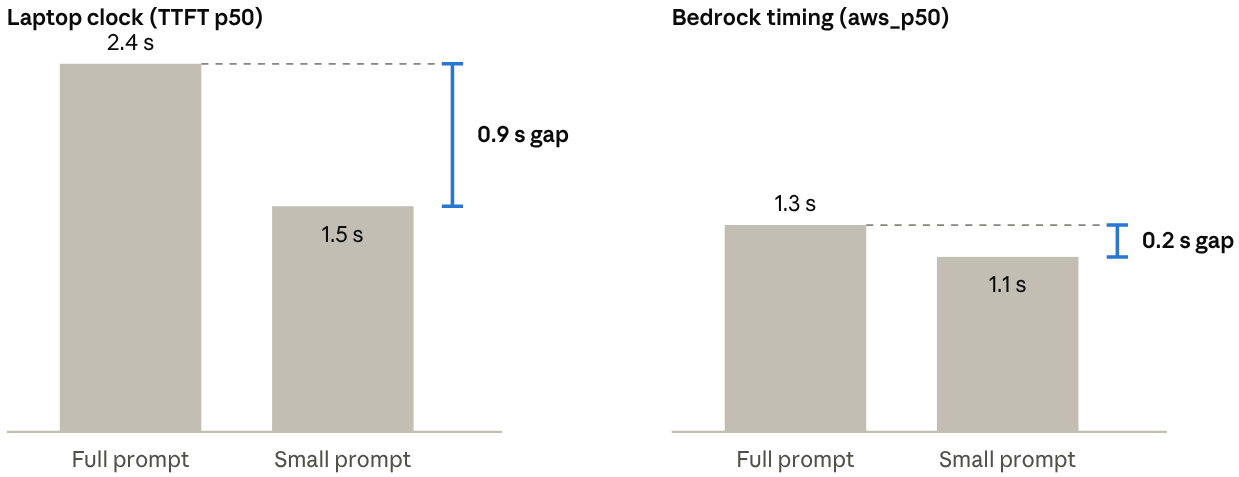
A slow minute lands on all six arms

run order · 6 arms x 3 runs

Two details matter more than they look:

- Server timing. Bedrock reports its own latencyMs. Your laptop's clock includes uploading a 70 KB prompt, which for us was a whole second.
- Provenance. Record the code version and settings on every row, so runs with different settings never get averaged together.

The big prompt costs 0.2 s on Bedrock's clock, not the 0.9 s the laptop shows



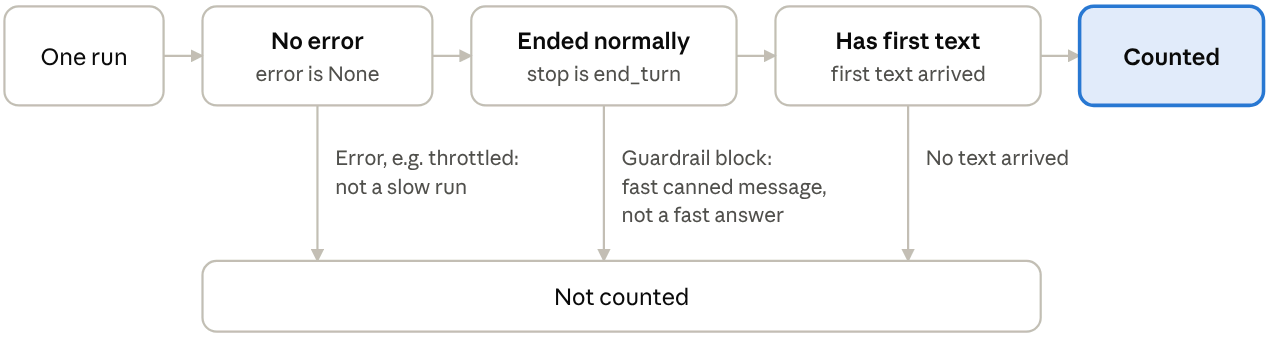
Guardrail off in both arms. Plain-answer call, medians over 15 valid runs per arm.

from the Step 6 report · p50 of 15 valid runs per arm, plain-answer call

Step 5: Count only valid runs, then report

A guardrail block streams a fast canned message. It's not a fast answer, so it must not count as one.

Only runs that finish cleanly with real text are counted



validity filter · 3 checks

```
def is_valid(row):
    # A guardrail block streams a fast canned message. It's not a fast
answer.
    return row["error"] is None and row["stop"] == "end_turn" and
row["first_text_ms"]

def pct(values, q):
    values = sorted(values)
    return values[max(0, math.ceil(q / 100 * len(values)) - 1)]   # nearest
rank; never round()

def report(rows):
    """Per arm: runs, valid, blocked, TTFT p50/p90, server-side estimate,
    time from first text to end of stream, output tokens."""
    ...
```

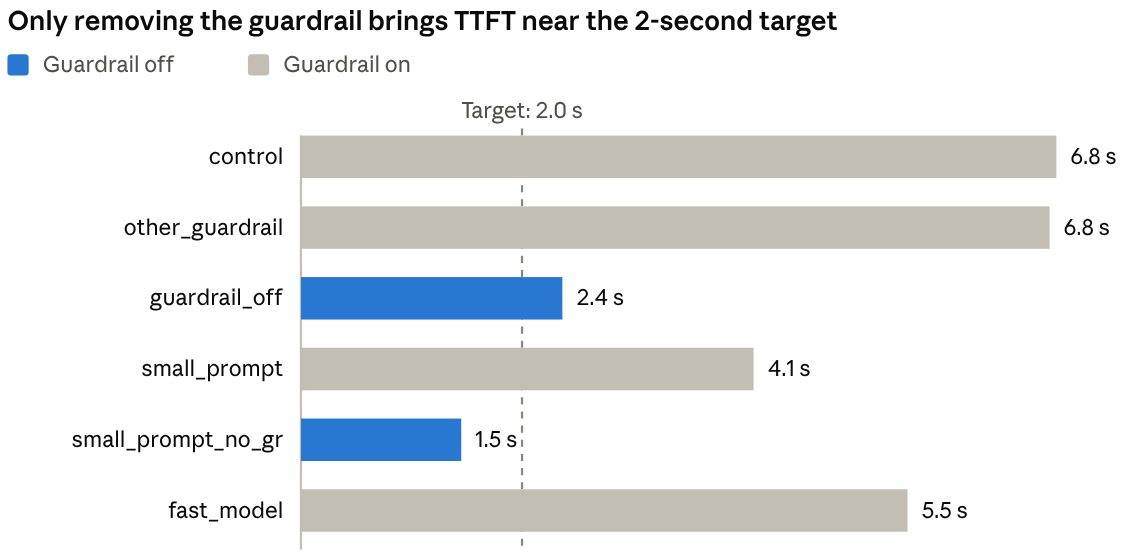
The report shows, per arm, how many runs each statistic was taken over.

Step 6: Read the report

These are 15 valid runs per arm for the plain-answer call. The aws_p50 column estimates TTFT inside AWS, from Bedrock's own timing.

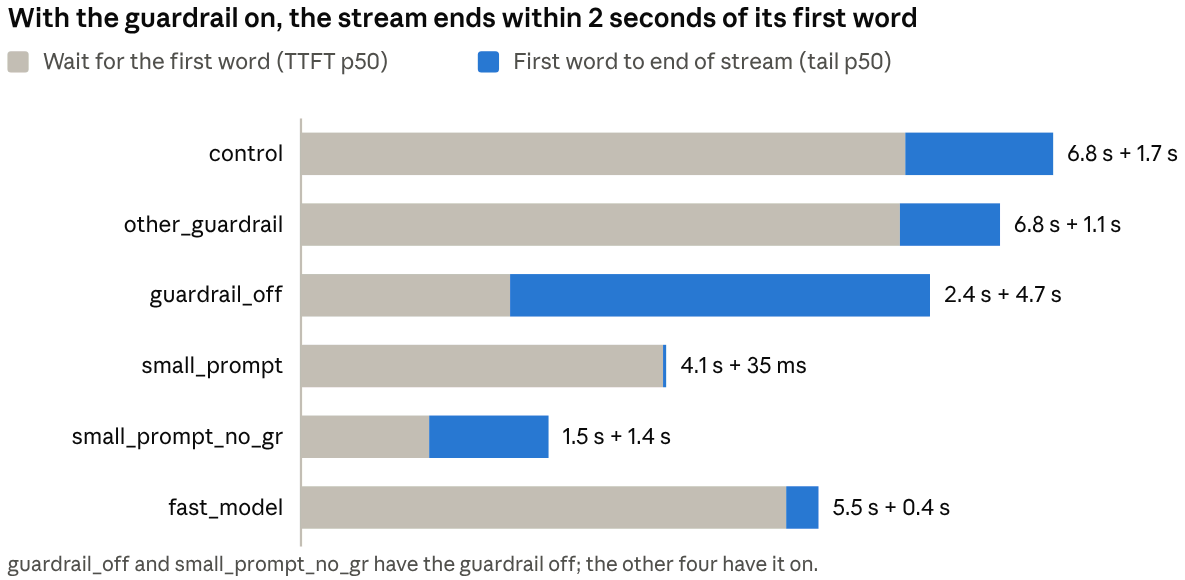
arm	used	blk	ttft_p50	ttft_p90	aws_p50	tail_p50	out_tok
control	15	0	6,835	8,553	5,693	1,672	488
other_guardrail	15	0	6,774	8,652	5,702	1,132	475
guardrail_off	15	0	2,365	2,578	1,329	4,749	508
small_prompt	15	0	4,095	7,516	3,713	35	148
small_prompt_no_gr	15	0	1,450	1,615	1,124	1,350	144
fast_model	15	0	5,488	5,669	4,006	365	279

Read it by subtracting one arm from another.



report · ttft_p50 by arm, 15 valid runs per arm, plain-answer call

Control minus guardrail_off: about 4.4 seconds. That's the guardrail. The tail_p50 column shows why. Without the guardrail, text starts at 1.3 s and streams for another 4.7 s. With it, the first text appears only 1–2 seconds before the stream ends. The synchronous guardrail check holds most of the reply back.



report · ttft_p50 and tail_p50 by arm, medians stacked, plain-answer call

Other_guardrail ≈ control. Both profiles cost the same, so tuning the profile won't help.

Guardrail_off minus small_prompt_no_gr: about 0.2 seconds. That's the 33,000-token prompt. On the laptop's clock the gap looked like 0.9 s, but most of that was upload. Trimming the prompt would have been a sprint spent on a fifth of a second.

On Bedrock's clock, 4.4 of the control's 5.7 seconds is the guardrail

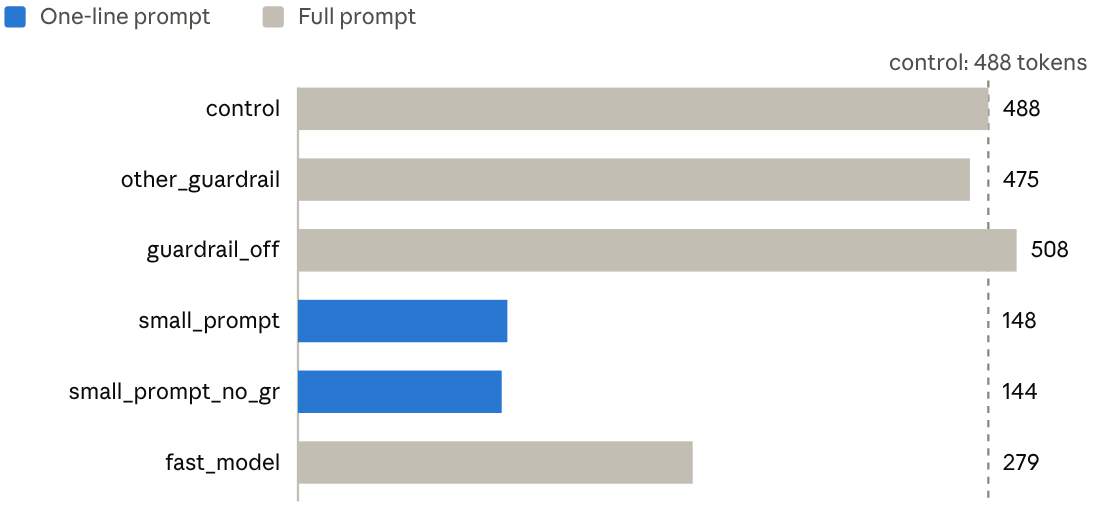


report · aws_p50 of control, guardrail_off, small_prompt_no_gr; differences computed

Fast_model: still about 4 s. Haiku writes faster but still waits behind the guardrail, so switching models doesn't reach the target.

Small_prompt looks fast for the wrong reason. It writes 148 tokens instead of 488, and under a synchronous guardrail a shorter reply is released sooner. The out_tok column catches that.

small_prompt only looks fast because its reply is a third as long



Output tokens per reply, plain-answer call.

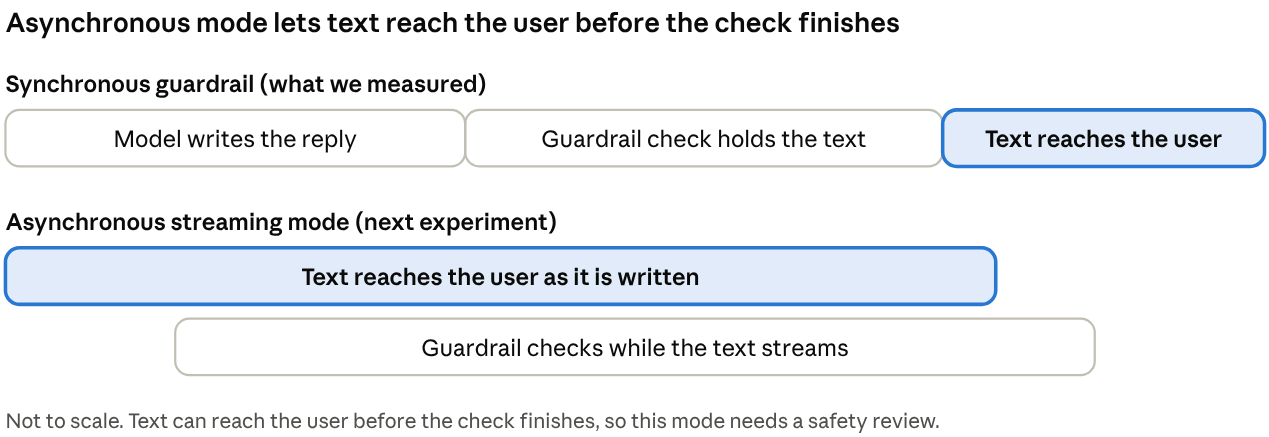
report · out_tok by arm, plain-answer call

The tool-result call told the same story: 4.9 s with the guardrail, 2.4 s without it, and a 1.1 s floor.

So the suspect wasn't the prompt or the model. It was how the output was checked.

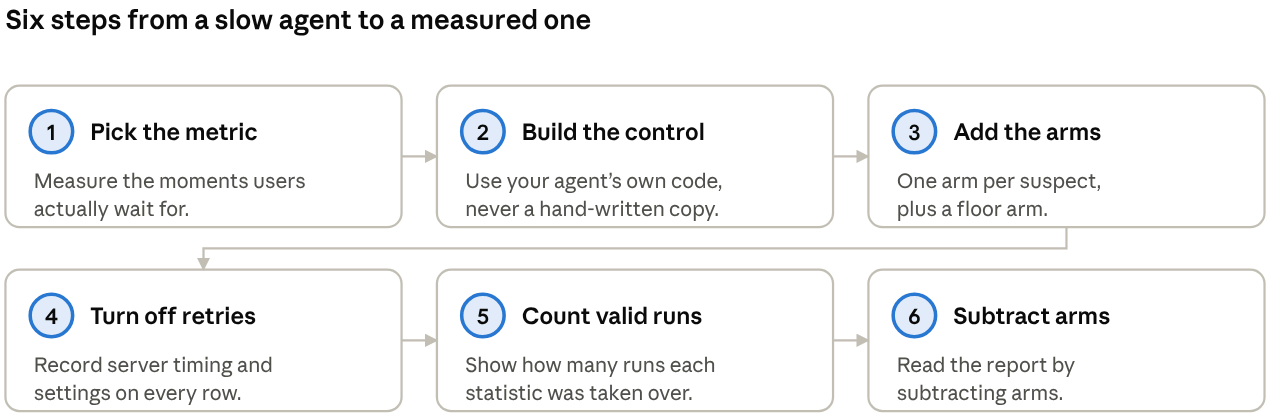
What we test next

That pointed us at the next experiment: the guardrail's asynchronous streaming mode, with a safety review, because in that mode text can reach the user before the check finishes.



sync vs async guardrail · not to scale

Starting MAT on your own agent



MAT checklist · 6 steps

1. Pick the metric and the moments users actually wait for.
2. Build the control with your agent's own code, never by hand.
3. Add one arm per suspect, plus a floor arm.
4. Turn off client retries and record server timing and settings on every row.
5. Count only valid runs, and show how many runs each statistic was taken over.

6. Read the report by subtracting arms.

Vibe coding gets your agent built. MAT shows you where its time goes.